Bloom

PARFUMS

Full-Stack Architecture Report

VERSION 3.0 — LIVE CODEBASE ANALYSIS

Prepared by: Senior Full-Stack Architect AI

Date: February 25, 2026

Scope: Next.js Frontend + Laravel Backend

Status: Blueprint — Ready for Implementation

Ground-truth analysis of the live codebase.
Covers 12 pages, 18 components, 13 data entities,
complete database DDL, full API contract, and implementation roadmap.

Full-Stack Architecture Report — Bloom Parfums

Prepared by: Senior Full-Stack Architect AI
Date: February 25, 2026
Scope: Complete analysis of `frontend/` (Next.js) + `backend/` (Laravel)
Version: 3.0 — Based on live codebase scan
Status: Architectural Blueprint — Ready for Implementation

> *This report is a ground-truth analysis of the actual codebase as it exists today. It documents what is built, what is missing, what is wrong, and the complete blueprint to bring this project to production. It supersedes all previous reports where conflicts arise.*

Table of Contents

---

1. Frontend Analysis — Pages & Routes

1.1 Route Map (12 Routes Identified)

Route	File	Render Strategy	Auth	Status
`/`	`app/page.tsx`	SSG	Public	✅ Complete
`/collection`	`app/collection/page.tsx`	CSR	Public	⚠️ Hardcoded data
`/product/[id]`	`app/product/[id]/page.tsx`	CSR	Public	⚠️ Mock data only
`/checkout`	`app/checkout/page.tsx`	CSR	Public	⚠️ No submission logic
`/success`	`app/success/page.tsx`	CSR	Public	⚠️ Static content

Route	File	Render Strategy	Auth	Status
`/track-order`	`app/track-order/page.tsx`	CSR	Public	⚠️ Form not wired
`/order-status`	`app/order-status/page.tsx`	CSR	Public	⚠️ Static timeline
`/feedback`	`app/feedback/page.tsx`	CSR	Public	⚠️ No submission
`/wishlist`	`app/wishlist/page.tsx`	CSR	Public	⚠️ Hardcoded list
`/login`	`app/login/page.tsx`	CSR	Public	⚠️ Targets removed endpoint
`/register`	`app/register/page.tsx`	CSR	Public	⚠️ Targets removed endpoint
`/dashboard`	`app/dashboard/page.tsx`	SSR	JWT cookie	⚠️ No user accounts planned

1.2 Page-by-Page Analysis

Route `/` — Homepage

Purpose: Marketing entry point. Brand identity, product discovery, social proof.
Render: SSG — no dynamic data, no auth. Correct choice.
User Actions: Browse hero, scan best sellers, explore categories, navigate to collection.
Sections Rendered (order):

1. `HeroSection` — hero banner with offer overlay
2. `BrandLogos` — brand logo strip (Boss, Prada, etc.)
3. `BestSellers` — carousel of 5 hardcoded products with prev/next arrows
4. `CategoriesSection` — category grid
5. `UniversSection` — brand universe / lifestyle section
6. `ValentinesSection` — seasonal promotional banner
7. `CustomerReviewsSection` — aggregate rating (4.5 / 2689 reviews) + testimonial cards

Data Required from Backend:

- Featured products (`is_featured = true`) for BestSellers
- Brand list for BrandLogos
- Category list for CategoriesSection
- Approved reviews (3–4 cards) for CustomerReviewsSection

Gap: All data is currently hardcoded. No API calls exist on this page.

Route `/collection` — Product Catalog

Purpose: Full product browsing experience with sidebar filters and product grid.
Render: `'use client'` — CSR. Appropriate given heavy filter interactivity.
User Actions: Filter by brand (searchable radio), price range (slider), gender (radio), category (radio), notes (star rating), promotions; toggle grid/list view; sort; paginate.
Sidebar Filters Identified:

- **Brand** — searchable radio list: Boss, Prada, Lancome, Dior, Chanel, Balenciaga, Versace, Sauvage (8 brands)
- **Price** — range slider with histogram visualization (80 MAD – 400 MAD)
- **Gender** — radio: Woman (110), Man (84), Child (20)
- **Category** — radio: Nouveautes, Visage, Corps, Parfums, Skincare, Marques
- **Notes** — star rating filter (Tout, 5.0, 4.0)
- **Promotions** — checkbox: Offre Speciales (92)

Product Grid: Renders `ProductCard` components, currently with hardcoded data.
Toolbar: Item count ("140 Produits"), grid/list toggle, sort dropdown, CartDrawer/ArrowLeftRight icons.

Data Required from Backend:

- Paginated products with filters (brand_id, category_id, gender, price_min, price_max, min_rating, on_sale)
- Brand list for filter sidebar
- Category list for filter sidebar

Gap: All product data is static arrays. Filters are purely visual (no state bound). Sort is not wired. Pagination is absent.

Route `/product/[id]` — Product Detail

Purpose: Full single product view. Primary conversion page.
Render: CSR (`'use client'`) — uses `params.id` for product lookup, but currently uses hardcoded mock data.
User Actions: Browse image gallery (thumbnails + main), select size (50ml, etc.), set quantity (± stepper), add to cart, toggle wishlist, share, expand/collapse accordion sections.
Accordion Sections:

- Description
- Ingredients
- Delivery information
- Customer reviews (with star breakdown)
- FAQ

State Managed Locally:

- `quantity` (default: 1)
- `selectedSize` (default: '50ml')
- `mainImage` (active gallery image)
- 5 accordion open/close flags
- FAQ open index

Icons Used: Heart, ShoppingCart, Share2, ChevronUp/Down, Truck, Clock, Banknote, Package, ChevronLeft/Right

Data Required from Backend:

- Product by slug/id: name, subtitle, description, price, originalPrice, rating, reviewCount, images[], sizes[], brand, category, ingredients, isBestSeller, isActive
- Product reviews (paginated, approved only)

Gap: Product data is a hardcoded object `{ id: params.id, name: 'SUGAR POP', ... }`. No API call. This is the most critical missing wiring in the entire frontend.

Route `/checkout` — Checkout

Purpose: Shipping address form + order summary + coupon + payment trigger.
Render: CSR (implied, no `'use client'` directive — but uses `defaultValue` attributes suggesting future state binding).
Shipping Fields: First Name, Last Name, Phone (country selector), City, Quartier (neighborhood), Zip Code, Address Line.
Shipping Methods (3 options):

- Free Shipping — 0 DH — Laayoune
- Région — 20 DH — Laayoune-Sakia el Hamra
- National — 35 DH — Tous les villes du maroc

Order Summary (right column): Cart items with thumbnail, name, variant, price; coupon code input (validated state shown in green); subtotal, shipping, coupon discount, total.
Coupon State: A "Coupon validé" state is mocked (green dashed border, green button). Not wired to API.

Data Required from Backend:

- Cart items (from Zustand store — not yet implemented)
- Shipping methods list
- Coupon validation endpoint
- Order creation endpoint (POST on form submit)

Gap: Form submits to nowhere. Coupon validation is purely visual. Cart items are hardcoded in the template. Shipping method selection has no onChange handler.

Route `/success` — Order Confirmation

Purpose: Post-checkout confirmation screen. Reassures the customer, provides order details, links to tracking.
Render: CSR (no directive, but static content).
Content: Success icon, "ORDER RECEIVED" heading, order ID (#LX-8921-Q), date, delivery address, total amount, "Confirmation Call" notice box, CTA buttons (Continue Shopping → `/`, Track My Order → `/track-order`).

Data Required from Backend:

- Order number, date, delivery address, total — from order creation response OR URL param lookup.

Gap: Completely static. In production, order details must be passed from the checkout response (via URL query params or session) and rendered dynamically.

Route `/track-order` — Order Lookup Form

Purpose: Entry point for guest order tracking. Customer enters order ID + phone.
User Actions: Enter `order_number` field, enter phone number, submit.
Submit Action: Currently `<Link href="/order-status">` — no actual form submission or API call.

Data Required from Backend:

- `GET /api/v1/orders/{order_number}/track` — validated by phone match.

Gap: Form is decoration. Submit is a `<Link>` tag. No validation, no API call, no error handling.

Route `/order-status` — Order Status Timeline

Purpose: Visual order tracking screen showing current delivery stage.
Content:

- Order header: ORDER #LX-8921-Q, "Package Delivered"
- Success banner: "Successfully Delivered" with gold CheckCircle icon
- 4-step timeline: Order Valid → Dispatched → Shipped via Livreur → Delivered
- Each step shows timestamp and location
- Action buttons: "Leave a Review →" `/feedback`, "Need Help ?"
- Right column: Shipment Contents (3 items with thumbnails, names, prices)

Data Required from Backend:

- Order with status_histories array: `[{ step, label, timestamp, location }]`
- Order items for the "Shipment Contents" panel

Gap: Fully static. The timeline is a hardcoded 4-step UI. No dynamic data binding. No SWR polling for real-time updates.

Route `/feedback` — Post-Purchase Rating

Purpose: Collect star ratings for ordered products. Opens `ReviewModal` on product click.
Render: `'use client'` — manages `isModalOpen` and `selectedProduct` state.
User Actions: View ordered products, click any product row → opens ReviewModal with that product's data.
Left Column: Decorative Bloom Parfums logo frame + 5 gold stars.
Right Column: Product list, each row showing:

- Thumbnail, product name, variant
- For rated products: "Rated ✓" badge + star fill + label (Excellent, Good)
- For unrated products: empty star row awaiting input

State Management:

```
const [isModalOpen, setIsModalOpen] = useState(false);
const [selectedProduct, setSelectedProduct] = useState({ name, desc, image });
```

Data Required from Backend:

- Ordered product list (from order number or session context)
- Review submission endpoint (POST /api/v1/reviews)

Gap: Product list is hardcoded (3 static products). Review submission in `ReviewModal` "Publish review" button has no handler. No PUT to update existing review.

Route `/wishlist` — Wishlist Page

Purpose: Display saved products from the `bloom_wishlist` cookie.
Render: `'use client'`.
Content: 8 hardcoded identical "Over Dose" products in a 5-column grid. Each card: wishlist heart, discount badge, image, name, rating, price, add-to-cart button.
Controls: Grid/list toggle, sort dropdown ("Relevance (Default)").

Data Required from Backend:

- `GET /api/v1/products/validate?ids=...` — to validate/resolve cookie IDs to real product data.

Gap: 8 duplicates of one hardcoded product. Cookie utility (`getWishlist`, `addToWishlist`) not implemented. No validation call to backend.

Route `/login` — Login Form

Purpose: JWT authentication form for user/admin login.
Render: `'use client'` — manages email, password, error, loading state.
Submit Action: Calls `authService.login()` → stores token via `setAuthToken()` → redirects to `/dashboard`.

Architectural Conflict: This page targets `POST /api/v1/auth/login` (public JWT). Under the v2.0 decisions, public user auth is removed. This page must be repurposed as `/admin/login` targeting `POST /api/v1/admin/auth/login`.

Route `/register` — Registration Form

Purpose: JWT user registration.
Data: name, email, password, password_confirmation.
Architectural Conflict: Public registration is explicitly dropped in v2.0. This route should be **removed entirely**. No customers create accounts.

Route `/dashboard` — Customer Dashboard

Purpose: Server-rendered user profile hub showing order count, wishlist count, loyalty points.
Render: SSR (`async` Server Component) — reads JWT from cookie, calls `serverFetch('/auth/me')`, pre-fetches user, renders `DashboardClient`.
Content: Welcome banner, stats grid (Orders: —, Wishlist: —, Loyalty points: 0), Sign out button.

Architectural Conflict: Customer accounts, loyalty points, and customer dashboard are all removed in v2.0. This route should be **removed or repurposed as `/admin/dashboard`**.

---

2. Frontend Analysis — Components Inventory

2.1 Layout Components

Component	File	Type	State	Dependencies
`Header`	`components/layout/Header.tsx`	Client	`cartCount, query, isCartOpen, isFilterOpen`	`CartDrawer`, `FilterModal`
`Footer`	`components/layout/Footer.tsx`	Server	None	Lucide icons
`Navbar`	`components/Navbar.tsx`	Client	Receives `user, onLogout`	Used in DashboardClient only

Header detail:

- Announcement bar (scrolling ticker)
- Logo (links to `/`)
- Search bar with filter icon (`onClick` → `setIsFilterOpen(true)`)
- Wishlist icon (`href="/wishlist"`)
- Cart icon (badge with `cartCount`, opens `CartDrawer`)
- Navigation: MEN, WOMEN, BEAUTY, SALE (red), GIFT SETS, NEW ARRIVALS, BRANDS
- Renders `CartDrawer` and `FilterModal` conditionally

2.2 Modal / Overlay Components

Component	Purpose	Props	State
`CartDrawer`	Right-side sliding cart	`isOpen, onClose`	`isMounted` (SSR guard)
`FilterModal`	Right-side filter drawer from search bar	`isOpen, onClose`	`isMounted, selectedBrand, selectedGender`
`ReviewModal`	Center modal — product review submission	`isOpen, onClose, productName, productDesc, productImage`	`isMounted, rating, hoveredRating`
`WishlistOverlay`	Full-screen overlay wishlist	`isOpen, onClose`	None (no state)

CartDrawer detail:

- Shows "YOUR CART (3 ITEMS)" — hardcoded
- Lists 2 products (SUGAR POP, OVER DOSE) with qty +/- and trash icon
- Summary: subtotal, shipping, coupon
- Coupon input field
- Total amount
- "Proceed to Checkout" button → `/checkout`

FilterModal detail:

- Brand section: search input + scrollable radio list (7 brands)
- Price section: histogram bars + range slider + min/max pill inputs
- Gender section: radio list (Woman, Man, Child)
- All sections have `ChevronUp` collapsible indicators

ReviewModal detail:

- Product image + name + desc in header
- Star rating (hover + click state)
- Textarea for review body
- Photo upload button (`Plus` icon)
- "Publish review ›" button (no handler)

2.3 Section Components (Homepage)

Component	Purpose	Data Source
`HeroSection`	Full-width hero banner	Static (local image)
`BrandLogos`	Brand logo strip	Static (hardcoded logos)
`BestSellers`	5-card product carousel with prev/next	Hardcoded array of 5 `ProductCardProps`
`CategoriesSection`	Category grid	Static
`UniversSection`	Brand universe lifestyle tiles	Static
`ValentinesSection`	Seasonal promotional banner	Static image `/Valentines-image.png`
`CustomerReviewsSection`	Rating summary (4.5/2689) + 3 testimonial cards	Static

2.4 Core UI Components

`ProductCard` (**`components/ui/ProductCard.tsx`**):

- Full TypeScript interface: `id, name, subtitle, description, price, originalPrice, rating, reviewCount, imageUrl, badge?, isBestSeller?`
- Client component — manages `wished` state (heart toggle, local only)
- Renders badge ("Save Up To 100 DH"), best-seller badge, star rating, prices
- Navigates to `/product/${id}` on click
- Heart button uses `e.preventDefault()` + `e.stopPropagation()` correctly

`DashboardClient` (**`components/DashboardClient.tsx`**):

- Receives `user: User` from server component
- Handles logout: calls `authService.logout()`, clears cookie, redirects to `/login`
- Renders stats grid (Orders, Wishlist, Loyalty points — all empty)
- Uses `Navbar` component

2.5 Services & Utilities

`services/api.ts` :

- Axios instance with `baseURL: process.env.NEXT_PUBLIC_API_URL`, `withCredentials: true`
- Request interceptor: attaches `Bearer {token}` from `js-cookie`
- Response interceptor: on 401 → attempts `POST /auth/refresh`, retries request, on failure → clears token + redirects to `/login`
- `authService.login()`, `authService.register()` — public JWT auth

`lib/auth.ts` :

- `setAuthToken(token)` — writes JWT to js-cookie (client-readable, NOT HttpOnly)
- `getAuthToken()` — reads from cookie
- `clearAuthToken()` — removes cookie
- `isAuthenticated()` — boolean check
- `serverFetch<T>(path, token)` — server-side fetch with JWT header

`types/index.ts`:

- `User` — id, name, email, role ('admin' | 'customer'), timestamps
- `Product` — id, name, slug, description, price, stock, image_url, category, timestamps
- `Order` — id, user_id, status enum, total, items[]
- `OrderItem` — id, order_id, product, quantity, unit_price
- `PaginatedResponse<T>` — data[], meta{}, links{}
- `ApiError` — message, errors{}

---

3. UI/UX Audit

3.1 Clarity of User Flow

Post-Purchase Flow (Excellent): The 5-step post-purchase journey is exceptionally well-designed:

```
Checkout → /success → /track-order → /order-status → /feedback
```

Each step is linked to the next. Breadcrumbs are present on all steps. The visual hierarchy (serif typography `Playfair Display`, gold `#cda873`, warm off-white `#f4ece3`) is consistent throughout.

Catalog to Checkout Flow (Broken): The path from product browsing to purchase is currently non-functional:

```
/collection → [no link to product detail]
/product/[id] → "Add to Cart" → CartDrawer → Checkout
```

The `ProductCard` correctly links to `/product/[id]` but the add-to-cart button has no global state binding. `CartDrawer` never updates.

Auth Flow (Contradictory):
`/login` → `/dashboard` exists and is functional architecturally (server-side auth check works), but the entire flow contradicts the v2.0 direction of zero customer accounts. These pages must be repurposed or removed.

3.2 Consistency of Components

Well-Consistent:

- Brand colors are rigidly consistent across all pages: `#4a403a` (dark brown), `#cda873` (gold), `#f4ece3` (warm off-white), `#fdf8f1` (light warm background)
- Typography: `font-serif` (Playfair Display) for headings/labels, `font-sans` (Inter) for body
- Button style: `rounded-sm`, `italic`, dark brown fill or ghost, hover with lighten

Inconsistencies Found:

1. `Navbar.tsx` (in DashboardClient) uses `brand-600` / `brand-50` Tailwind tokens — these are NOT defined in `tailwind.config.ts`. Will produce invisible/broken styling in dashboard.
2. `LoginPage` uses `form-input` class — not defined in global CSS. Form inputs will be unstyled.
3. `btn-secondary` CSS class used in DashboardClient — undefined.
4. `/track-order` page references `aura-gold` custom Tailwind color in `focus-within:border-aura-gold transition-colors` on the input wrapper — this color is defined in `tailwind.config` but used inconsistently

(some pages use hex `#cda873` directly, others use `aura-gold`).

5. `WishlistOverlay` component exists as a full-screen overlay AND `/wishlist` is a full page — both show identical content. One is redundant.

3.3 Component Reusability

High reuse score:

- `ProductCard` — used in BestSellers, Collection (implied), Wishlist
- `Header` + `Footer` — present on all 12 pages
- `SectionContainer` — used across multiple homepage sections

Missing reusable components identified:

1. **`OrderItemRow`** — the cart item UI (thumbnail + name + qty stepper + price + delete) is duplicated verbatim in: `CartDrawer`, `CheckoutPage` (right column), `OrderStatusPage` (right column), `OrderSuccessPage` (right column). Should be one shared component.
2. **`StarRating`** — star rendering logic exists in `ProductCard` (custom SVG), `BestSellers` (via ProductCard), `CustomerReviewsSection` (raw SVG inline), `ReviewModal` (Lucide Star), `FeedbackPage` (Lucide Star). Five different implementations for the same UI element.
3. **`OrderTimeline`** — the 4-step timeline in `/order-status` will need to be shared with the admin dashboard equivalent.
4. **`BreadcrumbNav`** — appears on 6 pages with different paths but the same rendering logic. Should be extracted.

3.4 Accessibility Issues

Issue	Severity	Location
Filter radio buttons use custom `<div>` elements, not `<input type="radio">`	High	FilterModal, CollectionPage sidebar
Star rating in ReviewModal uses `onClick` on `<Star>` icons without keyboard support	High	ReviewModal
Cart quantity +/- buttons have no `aria-label`	Medium	CartDrawer
Announcement bar scrolling text has no `aria-live` region	Low	Header
`<Link>` used as submit button in track-order form	High	`/track-order`
Images from Unsplash use generic alt text ("Sugar Pop") but external URLs will fail with `next/image` domain config	Medium	Multiple pages

3.5 Performance Risks

Risk	Type	Location	Recommendation
`Math.random()` in render in CollectionPage histogram	Re-render instability	`/collection` sidebar	Memoize or use fixed seed values
8 identical product items in wishlist (array duplication)	Memory/DOM waste	`/wishlist`, `WishlistOverlay`	Replace with real cookie data

Risk	Type	Location	Recommendation
`isMounted` pattern in CartDrawer, FilterModal, ReviewModal adds hydration delay per modal	Minor perf	All 3 drawers	Replace with `useEffect` portal or `suppressHydrationWarning`
No `loading.tsx` files in any App Router route	UX freeze on slow connections	All routes	Add route-level Suspense boundaries
`CustomerReviewsSection` has hard-coded product image URLs pointing to Google `lh3.googleusercontent.com`	External dep / breakage risk	Homepage	Migrate to own S3/CDN
`BestSellers` carousel has prev/next buttons that are decorative only (no scroll logic)	Non-functional UI	Homepage	Implement scroll with `useRef`

3.6 Missing UX Elements

Missing Element	Impact	Priority
Loading states on all API calls	High — blank screens on slow networks	P0
Empty states (empty cart, empty wishlist, no results)	High — broken UX with real data	P0
Error states on form submissions	High — silent failures	P0
Toast/notification system for add-to-cart, wishlist toggle	Medium	P1
Mobile hamburger navigation	High — nav is desktop-only visible	P0
Product image zoom on detail page	Low	P2
Loading skeleton for ProductCard	Medium	P1
404 and error pages (`not-found.tsx` , `error.tsx`)	Medium	P1

4. Functional Requirements Extraction

4.1 Core Public Features (Guest Access)

Feature	Evidence	Backend Needed
Browse product catalog with filters	`/collection` sidebar	`GET /products` with query params
View product detail with size variants	`/product/[id]`	`GET /products/{slug}`
Add to cart (client state)	`CartDrawer` , `ProductCard`	Zustand store only

Feature	Evidence	Backend Needed
Checkout as guest	`/checkout` form	`POST /orders`
Apply coupon code	Checkout coupon input	`POST /cart/coupon/validate`
Track order by order_number + phone	`/track-order` → `/order-status`	`GET /orders/{number}/track`
Save to wishlist (cookie)	Heart button in `ProductCard`	`GET /products/validate?ids=...`
View wishlist	`/wishlist`	Resolve cookie IDs to product data
Submit product review	`/feedback` → `ReviewModal`	`POST /reviews`
View product reviews	`/product/[id]` accordion	`GET /products/{slug}/reviews`

4.2 Post-Purchase Flow Features

Feature	Route	Status
Order confirmation display	`/success`	Static — needs dynamic data
Order tracking form	`/track-order`	Form not wired
Order timeline view	`/order-status`	Static — needs API
Product feedback submission	`/feedback`	No handler
Review modal submission	`ReviewModal`	No handler

4.3 Admin Features (Inferred from `ProductController` + Dashboard)

The frontend does not yet have an admin panel (`/admin/*`), but the backend `ProductController` already uses an `authorizeAdmin()` role check. Required admin capabilities inferred from the data model:

Feature	Backend Controller	Status
Product CRUD	`Admin\ProductController`	Partial (CRUD logic exists but needs refactor)
Brand CRUD	`Admin\BrandController`	Missing
Category CRUD	`Admin\CategoryController`	Missing
Coupon management	`Admin\CouponController`	Missing
Order management + status update	`Admin\OrderController`	Missing
Review moderation (approve/reject)	`Admin\ReviewController`	Missing
Admin login	`AdminAuthController`	Missing

4.4 Auth / Guest Decision Matrix

Feature	Access Model	Decision
Product browsing	Guest	Public — no token
Order creation	Guest	Public — no token
Order tracking	Guest	Public — order_number + phone match
Cart management	Guest	Zustand (in-memory, no backend)
Wishlist	Guest	Cookie (bloom_wishlist, no backend write)
Review submission	Guest	Public — rate limited
Admin panel	Admin only	Sanctum token in HttpOnly cookie
`/login`	Admin only	Repurpose as `/admin/login`
`/register`	None	**Remove** — no public registration
`/dashboard`	None	**Repurpose** as `/admin/dashboard`

4.5 Inferred Roles

Entity	Role	Rights
Visitor / Guest	Public	Browse, cart, checkout, track, review
Admin	Authenticated	Full CRUD on all entities + order management

No customer accounts. No customer roles.

---

5. Data Model Inference

5.1 Entity: `Admin`

Purpose: Single-row admin credential table for the store owner.
Attributes:

Field	Type	Constraints
id	bigint unsigned	PK, auto-increment
email	varchar(191)	NOT NULL, UNIQUE
password	varchar(255)	NOT NULL (bcrypt)
created_at	timestamp	nullable
updated_at	timestamp	nullable

Relationships: None (single entity, no FK relationships needed).

5.2 Entity: `Brand`

Purpose: Fragrance brands featured in the catalog (Boss, Prada, Lancome, Dior, etc.).
Inferred from: Collection sidebar Brand filter, BrandLogos section, FilterModal brand list.

Field	Type	Constraints
id	bigint unsigned	PK
name	varchar(100)	NOT NULL
slug	varchar(120)	NOT NULL, UNIQUE
logo_url	varchar(500)	nullable
created_at	timestamp	nullable
updated_at	timestamp	nullable

Relationships: `Brand` → 1-N → `Product`

5.3 Entity: `Category`

Purpose: Product classification (Nouveautes, Visage, Corps, Parfums, Skincare, Marques).
Inferred from: Collection sidebar Category filter.

Field	Type	Constraints
id	bigint unsigned	PK
name	varchar(100)	NOT NULL
slug	varchar(120)	NOT NULL, UNIQUE
parent_id	bigint unsigned	nullable, FK → categories.id
sort_order	tinyint unsigned	NOT NULL, default: 0
created_at	timestamp	nullable
updated_at	timestamp	nullable

Relationships: `Category` → self-referential (parent/child), `Category` → 1-N → `Product`

5.4 Entity: `Product`

Purpose: Core sellable entity. The center of the entire data model.
Inferred from: ProductCard, CollectionPage, CheckoutPage, ProductDetailPage, CartDrawer, OrderStatusPage.

Field	Type	Constraints
id	bigint unsigned	PK
brand_id	bigint unsigned	NOT NULL, FK → brands.id

Field	Type	Constraints
category_id	bigint unsigned	NOT NULL, FK → categories.id
name	varchar(255)	NOT NULL
slug	varchar(300)	NOT NULL, UNIQUE
subtitle	varchar(255)	nullable (e.g. "Body Butter")
description	text	NOT NULL
ingredients	text	nullable
gender	enum('man','woman','child','unisex')	NOT NULL
price	decimal(10,2)	NOT NULL
original_price	decimal(10,2)	nullable (for discount display)
stock	int unsigned	NOT NULL, default: 0
is_active	tinyint(1)	NOT NULL, default: 1
is_featured	tinyint(1)	NOT NULL, default: 0
created_at	timestamp	nullable
updated_at	timestamp	nullable
deleted_at	timestamp	nullable (soft delete)

Relationships:

- `Product` → N-1 → `Brand`
- `Product` → N-1 → `Category`
- `Product` → 1-N → `ProductImage`
- `Product` → 1-N → `ProductSize`
- `Product` → 1-N → `OrderItem`
- `Product` → 1-N → `Review`

5.5 Entity: `ProductImage`

Purpose: Multiple product images per product (gallery support).
Inferred from: ProductDetailPage (`images[]` array), single `image_url` on current model is insufficient.

Field	Type	Constraints
id	bigint unsigned	PK
product_id	bigint unsigned	NOT NULL, FK → products.id
url	varchar(500)	NOT NULL
alt	varchar(255)	nullable
sort_order	tinyint unsigned	NOT NULL, default: 0

Field	Type	Constraints
is_primary	tinyint(1)	NOT NULL, default: 0
created_at	timestamp	nullable

Relationships: `ProductImage` → N-1 → `Product`

5.6 Entity: `ProductSize`

Purpose: Size variants per product with optional per-size pricing.
Inferred from: ProductDetailPage `selectedSize` state (50ml, etc.), checkout item shows "Size 50ml".

Field	Type	Constraints
id	bigint unsigned	PK
product_id	bigint unsigned	NOT NULL, FK → products.id
label	varchar(50)	NOT NULL (e.g. "50ml", "100ml")
price_modifier	decimal(8,2)	NOT NULL, default: 0.00
stock	int unsigned	NOT NULL, default: 0
created_at	timestamp	nullable

Relationships: `ProductSize` → N-1 → `Product`

5.7 Entity: `ShippingMethod`

Purpose: Delivery options shown at checkout (Free/Région/National).
Inferred from: `/checkout` shipping method selector (3 cards with names, prices, zones).

Field	Type	Constraints
id	bigint unsigned	PK
name	varchar(100)	NOT NULL (e.g. "National")
description	varchar(255)	nullable (e.g. "Tous les villes du Maroc")
price	decimal(8,2)	NOT NULL
free_over	decimal(10,2)	nullable (free if order > X DH)
is_active	tinyint(1)	NOT NULL, default: 1
sort_order	tinyint unsigned	NOT NULL, default: 0

5.8 Entity: `Coupon`

Purpose: Discount codes validated server-side at checkout.
Inferred from: `/checkout` coupon input, `CartDrawer` coupon section, both show validated state.

Field	Type	Constraints
id	bigint unsigned	PK
code	varchar(50)	NOT NULL, UNIQUE
type	enum('fixed','percent')	NOT NULL
value	decimal(8,2)	NOT NULL
min_order_amount	decimal(10,2)	nullable
usage_limit	int unsigned	nullable (null = unlimited)
used_count	int unsigned	NOT NULL, default: 0
expires_at	timestamp	nullable
is_active	tinyint(1)	NOT NULL, default: 1
created_at	timestamp	nullable
updated_at	timestamp	nullable

5.9 Entity: `Order`

Purpose: Guest order record. Address absorbed as JSON fields — no addresses table.
Inferred from: `/checkout` form fields, `/success` order details, `/track-order` lookup, `/order-status` timeline.

Field	Type	Constraints
id	bigint unsigned	PK
order_number	varchar(20)	NOT NULL, UNIQUE, indexed (e.g. "LX-8921-Q7F")
shipping_method_id	bigint unsigned	NOT NULL, FK → shipping_methods.id
coupon_id	bigint unsigned	nullable, FK → coupons.id
first_name	varchar(100)	NOT NULL
last_name	varchar(100)	NOT NULL
phone	varchar(20)	NOT NULL
city	varchar(100)	NOT NULL
quartier	varchar(100)	nullable
zip_code	varchar(20)	nullable

Field	Type	Constraints
address_line	varchar(500)	NOT NULL
status	enum('pending','confirmed','dispatched','shipped','delivered','cancelled')	NOT NULL, default: 'pending'
subtotal	decimal(10,2)	NOT NULL
shipping_cost	decimal(10,2)	NOT NULL, default: 0
discount_amount	decimal(10,2)	NOT NULL, default: 0
total	decimal(10,2)	NOT NULL
notes	text	nullable
created_at	timestamp	nullable
updated_at	timestamp	nullable

Relationships:

- `Order` → N-1 → `ShippingMethod`
- `Order` → N-1 → `Coupon` (nullable)
- `Order` → 1-N → `OrderItem`
- `Order` → 1-N → `OrderStatusHistory`

5.10 Entity: `OrderItem`

Purpose: Individual line items within an order. Price locked at purchase time.
Inferred from: CartDrawer items, CheckoutPage order summary, OrderStatusPage shipment contents.

Field	Type	Constraints
id	bigint unsigned	PK
order_id	bigint unsigned	NOT NULL, FK → orders.id
product_id	bigint unsigned	NOT NULL, FK → products.id
size_label	varchar(50)	nullable (captured from size selection)
quantity	int unsigned	NOT NULL
unit_price	decimal(10,2)	NOT NULL (price at time of purchase)
created_at	timestamp	nullable

5.11 Entity: `OrderStatusHistory`

Purpose: Append-only log of status changes. Powers the 4-step timeline in `/order-status`.
Inferred from: OrderStatusPage timeline (Order Valid, Dispatched, Shipped, Delivered) each with timestamp and

location.

Field	Type	Constraints
id	bigint unsigned	PK
order_id	bigint unsigned	NOT NULL, FK → orders.id
status	varchar(50)	NOT NULL
label	varchar(255)	NOT NULL (e.g. "Order are dispatched")
location	varchar(255)	nullable (e.g. "Casablanca")
created_at	timestamp	nullable (= when this status occurred)

5.12 Entity: `Review`

Purpose: Customer-submitted product reviews with star rating, text body, and optional photos.
Inferred from: `ReviewModal` (star rating, textarea, photo upload), `CustomerReviewsSection`, ProductDetailPage reviews accordion, `/feedback` page.

Field	Type	Constraints
id	bigint unsigned	PK
product_id	bigint unsigned	NOT NULL, FK → products.id
order_number	varchar(20)	nullable (soft link to order, not FK)
reviewer_name	varchar(100)	NOT NULL
rating	tinyint unsigned	NOT NULL (1–5)
body	text	nullable
is_approved	tinyint(1)	NOT NULL, default: 0
approved_at	timestamp	nullable
created_at	timestamp	nullable
updated_at	timestamp	nullable

5.13 Entity: `ReviewImage`

Purpose: Photos attached to reviews via the "Upload Image" button in ReviewModal.
Inferred from: ReviewModal "PHOTOGRAPHY" section with `<Plus>` upload button.

Field	Type	Constraints
id	bigint unsigned	PK
review_id	bigint unsigned	NOT NULL, FK → reviews.id
url	varchar(500)	NOT NULL (Cloudflare CDN URL)

Field	Type	Constraints
created_at	timestamp	nullable

6. Database Schema

6.1 Complete DDL

```
-- ============================================================
-- BLOOM PARFUMS — Final Database Schema
-- Engine: MySQL 8.0+ (InnoDB)
-- Charset: utf8mb4
-- ============================================================

-- ── Admins ──────────────────────────────────────────────────
CREATE TABLE admins (
  id         BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  email      VARCHAR(191)    NOT NULL,
  password   VARCHAR(255)    NOT NULL,
  created_at TIMESTAMP       NULL DEFAULT NULL,
  updated_at TIMESTAMP       NULL DEFAULT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_admins_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── Brands ──────────────────────────────────────────────────
CREATE TABLE brands (
  id         BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  name       VARCHAR(100)    NOT NULL,
  slug       VARCHAR(120)    NOT NULL,
  logo_url   VARCHAR(500)    NULL,
  created_at TIMESTAMP       NULL DEFAULT NULL,
  updated_at TIMESTAMP       NULL DEFAULT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_brands_slug (slug)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── Categories ──────────────────────────────────────────────
CREATE TABLE categories (
  id         BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  parent_id  BIGINT UNSIGNED NULL,
  name       VARCHAR(100)    NOT NULL,
  slug       VARCHAR(120)    NOT NULL,
  sort_order TINYINT UNSIGNED NOT NULL DEFAULT 0,
  created_at TIMESTAMP       NULL DEFAULT NULL,
  updated_at TIMESTAMP       NULL DEFAULT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_categories_slug (slug),
  KEY idx_categories_parent (parent_id),
  CONSTRAINT fk_categories_parent
    FOREIGN KEY (parent_id) REFERENCES categories (id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── Products ────────────────────────────────────────────────
CREATE TABLE products (
  id             BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  brand_id       BIGINT UNSIGNED NOT NULL,
  category_id    BIGINT UNSIGNED NOT NULL,
  name           VARCHAR(255)    NOT NULL,
  slug           VARCHAR(300)    NOT NULL,
  subtitle       VARCHAR(255)    NULL,
  description    TEXT            NOT NULL,
  ingredients    TEXT            NULL,
  gender         ENUM('man','woman','child','unisex') NOT NULL DEFAULT 'unisex',
  price          DECIMAL(10,2)   NOT NULL,
  original_price DECIMAL(10,2)   NULL,
  stock          INT UNSIGNED    NOT NULL DEFAULT 0,
  is_active      TINYINT(1)      NOT NULL DEFAULT 1,
```

```
  is_featured    TINYINT(1)      NOT NULL DEFAULT 0,
  created_at     TIMESTAMP       NULL DEFAULT NULL,
  updated_at     TIMESTAMP       NULL DEFAULT NULL,
  deleted_at     TIMESTAMP       NULL DEFAULT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_products_slug (slug),
  KEY idx_products_brand (brand_id),
  KEY idx_products_category (category_id),
  KEY idx_products_active_featured (is_active, is_featured),
  KEY idx_products_gender (gender),
  KEY idx_products_price (price),
  CONSTRAINT fk_products_brand
    FOREIGN KEY (brand_id) REFERENCES brands (id) ON DELETE RESTRICT,
  CONSTRAINT fk_products_category
    FOREIGN KEY (category_id) REFERENCES categories (id) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── Product Images ──────────────────────────────────
CREATE TABLE product_images (
  id         BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  product_id BIGINT UNSIGNED NOT NULL,
  url        VARCHAR(500)    NOT NULL,
  alt        VARCHAR(255)    NULL,
  sort_order TINYINT UNSIGNED NOT NULL DEFAULT 0,
  is_primary TINYINT(1)      NOT NULL DEFAULT 0,
  created_at TIMESTAMP       NULL DEFAULT NULL,
  PRIMARY KEY (id),
  KEY idx_product_images_product (product_id),
  KEY idx_product_images_primary (product_id, is_primary),
  CONSTRAINT fk_product_images_product
    FOREIGN KEY (product_id) REFERENCES products (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── Product Sizes ───────────────────────────────────
CREATE TABLE product_sizes (
  id             BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  product_id     BIGINT UNSIGNED NOT NULL,
  label          VARCHAR(50)     NOT NULL,
  price_modifier DECIMAL(8,2)    NOT NULL DEFAULT 0.00,
  stock          INT UNSIGNED    NOT NULL DEFAULT 0,
  created_at     TIMESTAMP       NULL DEFAULT NULL,
  PRIMARY KEY (id),
  KEY idx_product_sizes_product (product_id),
  CONSTRAINT fk_product_sizes_product
    FOREIGN KEY (product_id) REFERENCES products (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── Shipping Methods ────────────────────────────────
CREATE TABLE shipping_methods (
  id          BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  name        VARCHAR(100)    NOT NULL,
  description VARCHAR(255)    NULL,
  price       DECIMAL(8,2)    NOT NULL,
  free_over   DECIMAL(10,2)   NULL,
  is_active   TINYINT(1)      NOT NULL DEFAULT 1,
  sort_order  TINYINT UNSIGNED NOT NULL DEFAULT 0,
  PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── Coupons ─────────────────────────────────────────
CREATE TABLE coupons (
```

```
  id               BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  code             VARCHAR(50)     NOT NULL,
  type             ENUM('fixed','percent') NOT NULL,
  value            DECIMAL(8,2)    NOT NULL,
  min_order_amount DECIMAL(10,2)   NULL,
  usage_limit      INT UNSIGNED    NULL,
  used_count       INT UNSIGNED    NOT NULL DEFAULT 0,
  expires_at       TIMESTAMP       NULL DEFAULT NULL,
  is_active        TINYINT(1)      NOT NULL DEFAULT 1,
  created_at       TIMESTAMP       NULL DEFAULT NULL,
  updated_at       TIMESTAMP       NULL DEFAULT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_coupons_code (code),
  KEY idx_coupons_active (is_active, expires_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── Orders ──────────────────────────────────────────────
CREATE TABLE orders (
  id                 BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  order_number       VARCHAR(20)     NOT NULL,
  shipping_method_id BIGINT UNSIGNED NOT NULL,
  coupon_id          BIGINT UNSIGNED NULL,
  first_name         VARCHAR(100)    NOT NULL,
  last_name          VARCHAR(100)    NOT NULL,
  phone              VARCHAR(20)     NOT NULL,
  city               VARCHAR(100)    NOT NULL,
  quartier           VARCHAR(100)    NULL,
  zip_code           VARCHAR(20)     NULL,
  address_line       VARCHAR(500)    NOT NULL,
  status             ENUM('pending','confirmed','dispatched','shipped','delivered','cancelled')
                                     NOT NULL DEFAULT 'pending',
  subtotal           DECIMAL(10,2)   NOT NULL,
  shipping_cost      DECIMAL(10,2)   NOT NULL DEFAULT 0.00,
  discount_amount    DECIMAL(10,2)   NOT NULL DEFAULT 0.00,
  total              DECIMAL(10,2)   NOT NULL,
  notes              TEXT            NULL,
  created_at         TIMESTAMP       NULL DEFAULT NULL,
  updated_at         TIMESTAMP       NULL DEFAULT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY uq_orders_order_number (order_number),
  KEY idx_orders_phone (phone),
  KEY idx_orders_status (status),
  KEY idx_orders_created (created_at),
  CONSTRAINT fk_orders_shipping_method
    FOREIGN KEY (shipping_method_id) REFERENCES shipping_methods (id) ON DELETE RESTRICT,
  CONSTRAINT fk_orders_coupon
    FOREIGN KEY (coupon_id) REFERENCES coupons (id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── Order Items ─────────────────────────────────────────
CREATE TABLE order_items (
  id         BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  order_id   BIGINT UNSIGNED NOT NULL,
  product_id BIGINT UNSIGNED NOT NULL,
  size_label VARCHAR(50)     NULL,
  quantity   INT UNSIGNED    NOT NULL,
  unit_price DECIMAL(10,2)   NOT NULL,
  created_at TIMESTAMP       NULL DEFAULT NULL,
  PRIMARY KEY (id),
  KEY idx_order_items_order (order_id),
  KEY idx_order_items_product (product_id),
```

```
  CONSTRAINT fk_order_items_order
    FOREIGN KEY (order_id) REFERENCES orders (id) ON DELETE CASCADE,
  CONSTRAINT fk_order_items_product
    FOREIGN KEY (product_id) REFERENCES products (id) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── Order Status Histories ──────────────────────────────
CREATE TABLE order_status_histories (
  id         BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  order_id   BIGINT UNSIGNED NOT NULL,
  status     VARCHAR(50)     NOT NULL,
  label      VARCHAR(255)    NOT NULL,
  location   VARCHAR(255)    NULL,
  created_at TIMESTAMP       NULL DEFAULT NULL,
  PRIMARY KEY (id),
  KEY idx_osh_order (order_id),
  CONSTRAINT fk_osh_order
    FOREIGN KEY (order_id) REFERENCES orders (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── Reviews ─────────────────────────────────────────────
CREATE TABLE reviews (
  id            BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  product_id    BIGINT UNSIGNED NOT NULL,
  order_number  VARCHAR(20)     NULL,
  reviewer_name VARCHAR(100)    NOT NULL,
  rating        TINYINT UNSIGNED NOT NULL,
  body          TEXT            NULL,
  is_approved   TINYINT(1)      NOT NULL DEFAULT 0,
  approved_at   TIMESTAMP       NULL DEFAULT NULL,
  created_at    TIMESTAMP       NULL DEFAULT NULL,
  updated_at    TIMESTAMP       NULL DEFAULT NULL,
  PRIMARY KEY (id),
  KEY idx_reviews_product_approved (product_id, is_approved),
  KEY idx_reviews_order_number (order_number),
  CONSTRAINT fk_reviews_product
    FOREIGN KEY (product_id) REFERENCES products (id) ON DELETE CASCADE,
  CONSTRAINT chk_reviews_rating CHECK (rating BETWEEN 1 AND 5)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ── Review Images ───────────────────────────────────────
CREATE TABLE review_images (
  id         BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  review_id  BIGINT UNSIGNED NOT NULL,
  url        VARCHAR(500)    NOT NULL,
  created_at TIMESTAMP       NULL DEFAULT NULL,
  PRIMARY KEY (id),
  KEY idx_review_images_review (review_id),
  CONSTRAINT fk_review_images_review
    FOREIGN KEY (review_id) REFERENCES reviews (id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

6.2 Entity Relationship Summary

```
admins                        (standalone)

brands ──────────────────────► products (1-N)
categories ──────────────────► products (1-N, parent_id self-ref)
products ────────────────────► product_images (1-N)
products ────────────────────► product_sizes  (1-N)
products ────────────────────► order_items    (1-N)
products ────────────────────► reviews        (1-N)

shipping_methods ────────────► orders  (1-N)
coupons ─────────────────────► orders  (1-N, nullable)
orders ──────────────────────► order_items           (1-N)
orders ──────────────────────► order_status_histories (1-N)

reviews ─────────────────────► review_images (1-N)
```

6.3 Dropped Tables vs. Current Backend

Current Backend	Proposed Schema	Action
`users`	**Dropped**	Remove — no customer accounts
`password_reset_tokens`	**Dropped**	Remove
`sessions`	**Dropped**	Admin uses stateless token
`products`	**Kept + Extended**	Add brand_id, category_id, gender, subtitle, original_price, is_featured, is_active
—	`admins`	**New**
—	`brands`	**New**
—	`categories`	**New**
—	`product_images`	**New**
—	`product_sizes`	**New**
—	`shipping_methods`	**New**
—	`coupons`	**New**
—	`orders`	**New**
—	`order_items`	**New**
—	`order_status_histories`	**New**
—	`reviews`	**New**
—	`review_images`	**New**

7. Laravel Backend Architecture

7.1 Current Backend State (Honest Assessment)

The current backend is a **skeleton** — not a partial implementation. It has:

- ✅ Correct framework choice (Laravel 11, PHP 8.2)
- ✅ Correct project structure pattern (Api/V1 namespace)
- ✅ Correct middleware pattern (ForceJsonResponse)
- ✅ Correct resource pattern (ProductResource)
- ✅ Auto-slug generation on Product model
- ✅ Soft deletes on User and Product
- ❌ Wrong auth library: `tymon/jwt-auth` — should be `laravel/sanctum`
- ❌ Only 2 entities (User, Product) instead of 13 required
- ❌ User model with customer role — completely replaced by Admin model
- ❌ Product model has `category` as free text — should be FK to categories
- ❌ No Order, Review, Coupon, Brand, Category, ShippingMethod models
- ❌ Authorization uses `$user->role !== 'admin'` check inline in controller — should be middleware
- ❌ `StoreProductRequest` validates `category` as a string — incorrect

7.2 Target Directory Structure

```
app/
├── Http/
│   ├── Controllers/
│   │   └── Api/
│   │       └── V1/
│   │           ├── Admin/
│   │           │   ├── AdminAuthController.php
│   │           │   ├── ProductController.php
│   │           │   ├── BrandController.php
│   │           │   ├── CategoryController.php
│   │           │   ├── CouponController.php
│   │           │   ├── OrderController.php
│   │           │   └── ReviewController.php
│   │           ├── ProductController.php        (public)
│   │           ├── BrandController.php          (public)
│   │           ├── CategoryController.php       (public)
│   │           ├── ShippingMethodController.php (public)
│   │           ├── OrderController.php          (public: create + track)
│   │           ├── ReviewController.php         (public: submit + list)
│   │           └── CouponController.php         (public: validate)
│   ├── Middleware/
│   │   ├── ForceJsonResponse.php   ✅ (exists)
│   │   ├── EnsureAdmin.php         (new)
│   │   └── SecurityHeaders.php     (new)
│   ├── Requests/
│   │   ├── Admin/
│   │   │   ├── LoginRequest.php
│   │   │   ├── StoreProductRequest.php
│   │   │   ├── UpdateProductRequest.php
│   │   │   ├── StoreBrandRequest.php
│   │   │   ├── StoreCategoryRequest.php
│   │   │   ├── StoreCouponRequest.php
│   │   │   └── UpdateOrderStatusRequest.php
│   │   ├── StoreOrderRequest.php   (guest checkout)
│   │   ├── StoreReviewRequest.php
│   │   └── ValidateCouponRequest.php
│   └── Resources/
│       ├── ProductResource.php      ✅ (extend)
│       ├── ProductDetailResource.php
│       ├── BrandResource.php
│       ├── CategoryResource.php
│       ├── ShippingMethodResource.php
│       ├── OrderResource.php
│       ├── OrderTrackResource.php
│       ├── ReviewResource.php
│       └── CouponResource.php
├── Models/
│   ├── Admin.php
│   ├── Brand.php
│   ├── Category.php
│   ├── Product.php           ✅ (extend)
│   ├── ProductImage.php
│   ├── ProductSize.php
│   ├── ShippingMethod.php
│   ├── Coupon.php
│   ├── Order.php
│   ├── OrderItem.php
│   ├── OrderStatusHistory.php
│   ├── Review.php
│   └── ReviewImage.php
```

```
├── Services/
│   ├── OrderService.php
│   ├── CouponService.php
│   ├── ReviewService.php
│   └── ProductSearchService.php
└── Jobs/
    ├── SendOrderConfirmationEmail.php
    └── ProcessReviewImage.php
```

7.3 Models

Why each layer is needed:

Models — Eloquent models define the data structure, relationships, mass-assignment protection, casts, and business-level boot hooks (e.g., auto-slug generation). Without clean models, every controller becomes a raw query mess.

```
// Admin.php — extends Authenticatable, implements HasApiTokens
class Admin extends Authenticatable {
    use HasApiTokens;    // Sanctum
    use HasFactory;
    protected $fillable = ['email', 'password'];
    protected $hidden   = ['password'];
}

// Product.php — extend existing, add new relationships
class Product extends Model {
    use SoftDeletes;
    protected $fillable = [
        'brand_id','category_id','name','slug','subtitle',
        'description','ingredients','gender','price','original_price',
        'stock','is_active','is_featured'
    ];

    public function brand():    BelongsTo { return $this->belongsTo(Brand::class); }
    public function category(): BelongsTo { return $this->belongsTo(Category::class); }
    public function images():   HasMany   { return $this->hasMany(ProductImage::class)->orderBy('s
ort_order'); }
    public function sizes():    HasMany   { return $this->hasMany(ProductSize::class); }
    public function reviews():  HasMany   { return $this->hasMany(Review::class)->where('is_approv
ed', true); }
}

// Order.php
class Order extends Model {
    protected $fillable = [...all address fields..., 'order_number', 'status', 'subtotal', 'shippi
ng_cost', 'discount_amount', 'total'];

    public function items():           HasMany   { return $this->hasMany(OrderItem::class); }
    public function statusHistories(): HasMany   { return $this->hasMany(OrderStatusHistory::clas
s)->orderBy('created_at'); }
    public function shippingMethod():  BelongsTo { return $this->belongsTo(ShippingMethod::class);
}
    public function coupon():          BelongsTo { return $this->belongsTo(Coupon::class)->withTra
shed(); }

    // Generate unique order number like LX-8921-Q7F
    protected static function booted(): void {
        static::creating(function (Order $order) {
            $order->order_number = strtoupper('LX-' . random_int(1000,9999) . '-' . Str::random
(3));
        });
    }
}
```

7.4 Services (Business Logic)

Why Services: Controllers must be thin (receive request → delegate → return response). All business logic with multi-step operations, validations, and DB transactions belongs in a Service class.

`OrderService` — The most complex service. Responsibilities:

1. Validate all product IDs and sizes exist
2. Resolve prices from DB (never trust frontend-sent prices)
3. Check stock levels (throw `STOCK_INSUFFICIENT` if any item fails)

4. Atomically decrement stock (DB transaction)
5. Validate and apply coupon (delegates to CouponService)
6. Calculate: subtotal, shipping cost, discount, total
7. Create `Order` + `OrderItem` records
8. Create initial `OrderStatusHistory` entry (status: pending)
9. Dispatch `SendOrderConfirmationEmail` job to queue
10. Return the order

`CouponService` — Responsibilities:

1. Look up coupon by code
2. Check `is_active`, `expires_at`, `usage_limit`
3. Check `min_order_amount`
4. Calculate discount (fixed DH or percent)
5. On apply: increment `used_count`
6. Return `{ valid: bool, discount: decimal, message: string }`

`ReviewService` — Responsibilities:

1. Validate product exists and accepts reviews
2. Store review (unapproved by default)
3. Process uploaded images (dispatch `ProcessReviewImage` per image)
4. Invalidate review cache for this product slug

`ProductSearchService` — Responsibilities:

1. Build Eloquent query from filter params (brand_id, category_id, gender, price_min, price_max, min_rating, on_sale, search)
2. Apply eager loading (brand, images, sizes)
3. Handle sorting (price_asc, price_desc, newest, rating)
4. Return paginated collection

7.5 Requests (Validation)

Why FormRequest: Centralized, typed validation that runs before any controller code executes. Fails fast with a structured 422 response without touching business logic.

Key request classes:

- `StoreOrderRequest` — validates all 7 address fields, shipping_method_id, items[] array, each item's product_id and quantity
- `StoreReviewRequest` — validates product_id, reviewer_name (required), rating (integer 1-5), body (nullable, max 2000), images (max 3, max 5MB each, MIME validation from file content)
- `Admin\StoreProductRequest` — validates brand_id (exists in brands), category_id (exists in categories), gender (enum), price (numeric, min 0), original_price (nullable, numeric), stock (integer, min 0), images[] (uploaded files), sizes[] structure

7.6 Resources (Response Shaping)

Why Resources: Prevent field over-exposure. Control the exact JSON shape returned. Decouple DB column names from API field names.

```
// ProductResource (public listing) — lean response for grids
class ProductResource extends JsonResource {
    return [
        'id', 'name', 'slug', 'subtitle', 'price', 'original_price',
        'rating' => $this->reviews()->avg('rating'),
        'review_count' => $this->reviews()->count(),
        'is_featured', 'gender',
        'brand' => ['id', 'name', 'slug'],
        'category' => ['id', 'name', 'slug'],
        'primary_image' => $this->images->where('is_primary', true)->first()?->url,
    ];
}

// ProductDetailResource (single product) — full response for detail page
class ProductDetailResource extends JsonResource {
    // All ProductResource fields PLUS:
    // description, ingredients, sizes[], images[] (full gallery), approved_reviews (4 latest)
}

// OrderTrackResource — public tracking response (no sensitive data)
class OrderTrackResource extends JsonResource {
    return [
        'order_number', 'status', 'estimated_delivery',
        'status_histories' => OrderStatusHistoryResource::collection($this->statusHistories),
        'items_count' => $this->items->count(),
        // DO NOT expose: full address, phone, total
    ];
}
```

7.7 Middleware

`EnsureAdmin` — Guards all `/admin/*` routes after Sanctum token verification:

```
class EnsureAdmin {
    public function handle(Request $request, Closure $next): Response {
        if (!$request->user() instanceof Admin) {
            return response()->json(['message' => 'Forbidden'], 403);
        }
        return $next($request);
    }
}
```

`SecurityHeaders` — Injects security headers on every response:

```
// X-Content-Type-Options: nosniff
// X-Frame-Options: DENY
// Referrer-Policy: strict-origin-when-cross-origin
// Permissions-Policy: camera=(), microphone=()
```

`ForceJsonResponse` ✅ Already exists — forces `Accept: application/json` on every request so Laravel never returns HTML error pages.

7.8 Auth — Migration from JWT to Sanctum

Current state: `tymon/jwt-auth ^2.1` — JWT tokens stored in `js-cookie` (client-readable).
Target state: `laravel/sanctum` — Sanctum token stored in `HttpOnly, Secure, SameSite=Strict` cookie.

Migration steps:

1. `composer remove tymon/jwt-auth`
2. `composer require laravel/sanctum`
3. `php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"`
4. Replace `User implements JWTSubject` with `Admin extends Authenticatable` + `use HasApiTokens`
5. Replace `JWTAuth::fromUser()` with `$admin->createToken('admin-session')->plainTextToken`
6. Set token in `Set-Cookie` header (HttpOnly, Secure, SameSite=Strict)
7. Configure Sanctum stateful domains in `config/sanctum.php`

7.9 Queues

Two jobs required:

`SendOrderConfirmationEmail` — Triggered immediately after order creation. Sends order summary to customer's phone (WhatsApp / SMS) or email. Queue: `default`. Retries: 3.

`ProcessReviewImage` — Triggered for each uploaded review image. Resizes to 800px wide, converts to WebP, uploads to S3 via `Storage::disk('s3')`. Queue: `media`. Retries: 3.

---

8. API Contract Design

8.1 Public Endpoints

`GET /api/v1/products`
List products with filtering and sorting.
Auth: None | Cache: SWR 5 min

Query params:

```
brand_id      integer   optional
category_id   integer   optional
gender        string    optional (man|woman|child|unisex)
price_min     numeric   optional
price_max     numeric   optional
min_rating    numeric   optional (1–5)
on_sale       boolean   optional
search        string    optional (min 3 chars)
sort          string    optional (newest|price_asc|price_desc|rating)
page          integer   default: 1
per_page      integer   default: 20, max: 60
```

Response `200`:

```
{
  "data": [
    {
      "id": 1, "name": "SUGAR POP", "slug": "sugar-pop-abc123",
      "subtitle": "Body Butter", "price": 140.00, "original_price": 200.00,
      "rating": 4.9, "review_count": 180, "is_featured": true, "gender": "woman",
      "brand": { "id": 2, "name": "Boss", "slug": "boss" },
      "category": { "id": 3, "name": "Corps", "slug": "corps" },
      "primary_image": "https://cdn.bloomparfums.ma/products/sugar-pop.webp"
    }
  ],
  "meta": { "current_page": 1, "last_page": 7, "per_page": 20, "total": 140 },
  "links": { "first": "...", "last": "...", "prev": null, "next": "..." }
}
```

Errors: No errors for list endpoints — empty `data: []` for no results.

`GET /api/v1/products/{slug}`

Single product with full detail.

Auth: None | Cache: ISR 30 min

Response `200`:

```
{
  "data": {
    "id": 1, "name": "SUGAR POP", "slug": "sugar-pop-abc123",
    "subtitle": "Body Butter", "description": "...", "ingredients": "...",
    "price": 140.00, "original_price": 200.00,
    "rating": 4.9, "review_count": 180,
    "is_featured": true, "is_active": true, "gender": "woman",
    "brand": { "id": 2, "name": "Boss", "slug": "boss", "logo_url": "..." },
    "category": { "id": 3, "name": "Corps", "slug": "corps" },
    "images": [
      { "url": "...", "alt": "...", "is_primary": true },
      { "url": "...", "alt": "...", "is_primary": false }
    ],
    "sizes": [
      { "id": 1, "label": "50ml", "price_modifier": 0.00, "stock": 15 },
      { "id": 2, "label": "100ml", "price_modifier": 40.00, "stock": 8 }
    ]
  }
}
```

Errors:

- `404` → `{ "message": "Product not found.", "code": "ORDER_NOT_FOUND" }`
- `410` → `{ "message": "Product unavailable.", "code": "PRODUCT_UNAVAILABLE" }` (when is_active: false)

`GET /api/v1/products/validate?ids=12,47,103`

Validate wishlist cookie IDs.

Auth: None | Cache: None

Response `200`:

```
{
  "data": [
    { "id": 12, "active": true,  "name": "SUGAR POP", "price": 140, "primary_image": "..." },
    { "id": 47, "active": false, "name": null, "price": null, "primary_image": null },
    { "id": 103,"active": true,  "name": "OVER DOSE", "price": 100, "primary_image": "..." }
  ]
}
```

`GET /api/v1/brands`

Full brand list for filter sidebar.

Auth: None | Cache: Redis 6h

Response `200`:


```
{
  "data": [
    { "id": 1, "name": "Boss", "slug": "boss", "product_count": 12 },
    { "id": 2, "name": "Prada", "slug": "prada", "product_count": 58 }
  ]
}
```

`GET /api/v1/categories`

Category tree (with child nesting).

Auth: None | Cache: Redis 6h

Response `200`:


```
{
  "data": [
    {
      "id": 1, "name": "Nouveautes", "slug": "nouveautes", "product_count": 119,
      "children": []
    },
    {
      "id": 2, "name": "Corps", "slug": "corps", "product_count": 28,
      "children": [
        { "id": 6, "name": "Body Butter", "slug": "body-butter", "product_count": 10 }
      ]
    }
  ]
}
```

`GET /api/v1/shipping-methods`

Available shipping options for checkout.

Auth: None | Cache: Redis 24h

Response `200`:

```
{
  "data": [
    { "id": 1, "name": "Free Shipping", "description": "Laayoune", "price": 0.00, "free_over": nul
l },
    { "id": 2, "name": "Région", "description": "Laayoune-Sakia el Hamra", "price": 20.00, "free_o
ver": null },
    { "id": 3, "name": "National", "description": "Tous les villes du Maroc", "price": 35.00, "fre
e_over": 590.00 }
  ]
}
```

`POST /api/v1/cart/coupon/validate`

Validate a coupon code against a cart subtotal.
Auth: None | Rate limit: 20 req/min

Request:


```
{ "code": "PROMO10", "subtotal": 760.00 }
```


Response `200` (valid):


```
{
  "data": {
    "valid": true,
    "code": "PROMO10",
    "discount_type": "fixed",
    "discount_value": 40.00,
    "discount_amount": 40.00,
    "message": "Coupon validé — you save 40 DH"
  }
}
```


Response `200` (invalid):


```
{
  "data": {
    "valid": false,
    "message": "This coupon has expired.",
    "code": "COUPON_EXPIRED"
  }
}
```

`POST /api/v1/orders`

Create a guest order. Core checkout submission.
Auth: None | Rate limit: 10 req/min

Request:

```
{
  "first_name": "Ayoub",
  "last_name": "Laghzal",
  "phone": "+212611955060",
  "city": "Casablanca",
  "quartier": "Hay Hassani",
  "zip_code": "20230",
  "address_line": "N° 10, Rue XYZ, Appt 3",
  "shipping_method_id": 3,
  "coupon_code": "PROMO10",
  "items": [
    { "product_id": 1, "size_label": "50ml", "quantity": 1 },
    { "product_id": 2, "size_label": "50ml", "quantity": 3 }
  ]
}
```

Response `201`:

```
{
  "data": {
    "order_number": "LX-8921-Q7F",
    "status": "confirmed",
    "subtotal": 760.00,
    "shipping_cost": 35.00,
    "discount_amount": 40.00,
    "total": 755.00,
    "items": [
      { "product_id": 1, "name": "SUGAR POP", "size_label": "50ml", "quantity": 1, "unit_price": 1
40.00 },
      { "product_id": 2, "name": "OVER DOSE", "size_label": "50ml", "quantity": 3, "unit_price": 1
00.00 }
    ],
    "estimated_delivery": "2026-02-28"
  }
}
```

Errors:

- `409` → `{ "message": "Insufficient stock for SUGAR POP.", "code": "STOCK_INSUFFICIENT" }`
- `422` → validation errors for required fields
- `410` → `{ "message": "Coupon code is expired.", "code": "COUPON_EXPIRED" }`

`GET /api/v1/orders/{order_number}/track`

Guest order tracking. Public, rate-limited.

Auth: None | Rate limit: 10 req/min

Query: `?phone=+212611955060` (required — validated server-side)

Response `200`:

```
{
  "data": {
    "order_number": "LX-8921-Q7F",
    "status": "delivered",
    "status_histories": [
      { "label": "Order Valid", "created_at": "2026-10-22T10:30:00Z", "location": null },
      { "label": "Order are dispatched", "created_at": "2026-10-23T12:30:00Z", "location": null },
      { "label": "Shipped via Livreur", "created_at": "2026-10-23T17:30:00Z", "location": null },
      { "label": "Delivered", "created_at": "2026-10-25T18:00:00Z", "location": "Casablanca" }
    ],
    "items_count": 3
  }
}
```

Errors:

- `404` → `{ "message": "Order not found.", "code": "ORDER_NOT_FOUND" }` (same response whether not found or phone mismatch — prevent enumeration)

`POST /api/v1/reviews`

Submit a product review from the feedback page.

Auth: None | Rate limit: 5 req/min

Request (multipart/form-data for image support):

```
product_id    integer   required
order_number  string    optional
reviewer_name string    required
rating        integer   required (1–5)
body          string    optional (max 2000 chars)
images[]      file[]    optional (max 3, max 5MB each, jpeg/png/webp)
```

Response `201`:

```
{
  "data": {
    "id": 42,
    "reviewer_name": "Ayoub L.",
    "rating": 5,
    "body": "Describe the notes...",
    "is_approved": false,
    "message": "Thank you for your review. It will be published after moderation."
  }
}
```

`GET /api/v1/products/{slug}/reviews`

Approved reviews for a product (shown in product detail accordion).

Auth: None | Cache: Redis 15 min

Response `200`:

```
{
  "data": [
    { "id": 1, "reviewer_name": "Zineb E.", "rating": 5, "body": "...", "created_at": "2024-05-16", "images": [] }
  ],
  "meta": { "average_rating": 4.9, "total_reviews": 180 }
}
```

8.2 Admin Endpoints

All require `admin_token` cookie (Sanctum HttpOnly). All rate-limited to 300 req/min.

`POST /api/v1/admin/auth/login`

Request: `{ "email": "admin@bloomparfums.ma", "password": "..." }`
Response `200`: `{ "message": "Authenticated" }` + `Set-Cookie: admin_token=...; HttpOnly; Secure; SameSite=Strict; Max-Age=86400`
Errors: `401` on invalid credentials, `429` on brute-force (+5 attempts/hour)

`POST /api/v1/admin/auth/logout`

Response `200`: `{ "message": "Logged out" }` + clears `admin_token` cookie

`GET/POST/PUT/DELETE /api/v1/admin/products`

Full product CRUD including image upload, size management.

`GET/POST/PUT/DELETE /api/v1/admin/brands`
`GET/POST/PUT/DELETE /api/v1/admin/categories`
`GET/POST/PUT/DELETE /api/v1/admin/coupons`

Standard CRUD for catalog management entities.

`GET /api/v1/admin/orders`

Paginated order list with filters (status, date range, search by order_number or phone).

`PATCH /api/v1/admin/orders/{id}/status`

Request: `{ "status": "shipped", "label": "Shipped via Livreur", "location": "Casablanca" }`
Action: Updates `orders.status`, appends to `order_status_histories`.

`GET /api/v1/admin/reviews`

All reviews (pending + approved). Filterable by `is_approved`, `product_id`.

`PATCH /api/v1/admin/reviews/{id}/approve`

Request: `{ "approved": true }`
Action: Sets `is_approved`, sets `approved_at`, invalidates review cache.

8.3 Eliminated Endpoints (From Current Backend)

Current Endpoint	Status	Reason
`POST /api/v1/auth/register`	**Remove**	No public registration
`POST /api/v1/auth/login` (public)	**Repurpose** → `POST /api/v1/admin/auth/login`	Admin only
`POST /api/v1/auth/refresh`	**Remove**	Sanctum stateless; on expiry → redirect to /admin/login
`GET /api/v1/auth/me`	**Repurpose** → admin-only `/api/v1/admin/auth/me`	
`GET /api/v1/users`	**Remove**	User model dropped

9. Security & Scalability Notes

9.1 Authentication Architecture Gap

Current: JWT (tymon/jwt-auth) stored in `js-cookie`. Token is JavaScript-readable → XSS can steal it.
Target: Sanctum token stored in `HttpOnly, Secure, SameSite=Strict` cookie. JavaScript-blind.

This is a **hard requirement**, not a recommendation. The current setup has a direct XSS token theft vector.

9.2 Rate Limiting Strategy

```
Public catalog (GET endpoints):    120 req/min per IP
Order creation (POST /orders):      10 req/min per IP
Order tracking (GET /orders/track): 10 req/min per IP
Review submission:                   5 req/min per IP
Coupon validation:                  20 req/min per IP
Admin login:                         5 req/min per IP
  └── Hard block after 10 failures/hour (store in Redis)
Admin endpoints (authenticated):   300 req/min per token
```

Laravel 11 throttle middleware: `throttle:120,1` (120 per 1 minute).
Admin login brute-force: custom `LoginThrottle` middleware using Redis key `admin_login_fails:{ip}`.

9.3 Validation Risks

Risk	Current State	Fix
Frontend prices sent to backend	CartDrawer hardcodes prices	**Server ALWAYS resolves prices from DB** — never trust client prices
Product IDs in order not validated	No order endpoint exists yet	OrderService validates every product_id against DB before creating order
Coupon applied on frontend only	Visual state only	Backend always re-validates coupon on order create
Stock not checked	No endpoint	OrderService wraps stock decrement in DB transaction
Image MIME validated from extension	`image_url` uses URL, no upload	Use `Storage::mimeType()` on actual file content, not extension

9.4 N+1 Query Risks

ProductController@index without eager loading → each card in a 20-product grid triggers N queries for brand, category, images.
Fix: Always eager load in collections:

```
Product::with(['brand', 'category', 'images' => fn($q) => $q->where('is_primary', true)])->paginate(20);
```

OrderController@index (admin) → items per order creates N+1 per order.
Fix:

```
Order::with(['items.product', 'shippingMethod'])->paginate();
```

ReviewController@index → review images per review creates N+1.
Fix: `Review::with('images')->get()`

Enable Laravel Debugbar in development to catch N+1 before they reach production.

9.5 Caching Strategy

Data	Mechanism	TTL	Invalidation Trigger
Featured products	`Cache::remember('products.featured', 3600)`	1 hour	Admin updates any product
Brand list	`Cache::remember('brands.all', 21600)`	6 hours	Admin updates any brand
Category tree	`Cache::remember('categories.tree', 21600)`	6 hours	Admin updates any category
Product detail	`Cache::remember("product.{$slug}", 1800)`	30 min	Admin updates that product
Approved reviews	`Cache::remember("reviews.{$slug}", 900)`	15 min	Admin approves a review for that product

Data	Mechanism	TTL	Invalidation Trigger
Shipping methods	`Cache::remember('shipping_methods', 86400)`	24 hours	Admin updates shipping methods

Use `Cache::tags(['products'])` to flush all product-related keys atomically on any catalog mutation.

9.6 File Upload Security

1. **Validate from content, not extension:** Use `Validator::make(['image' => $file], ['image' => 'mimes:jpeg,png,webp'])` — Laravel validates from actual MIME type, not filename.
2. **Max size:** 5 MB per file, enforced in `StoreReviewRequest`.
3. **Storage:** `Storage::disk('s3')` only. Never serve files from `public/` for user uploads.
4. **Processing:** Resize to 800px wide, convert to WebP via `ProcessReviewImage` queued job (not synchronously in the request lifecycle).
5. **Public URL:** Serve via Cloudflare CDN. S3 bucket has no public access policy. CDN origin pulls from S3.

9.7 Frontend Security Gaps

Gap	Risk	Fix
`brand-600`, `btn-secondary`, `form-input` undefined CSS classes	UI broken in dashboard/login	Define in `globals.css` or Tailwind config
`aura-gold` Tailwind token used inconsistently alongside raw hex	Future color drift	Standardize: use token everywhere, remove raw hex
No `middleware.ts` for admin route protection	Admin panel accessible without auth	Add Next.js Middleware: check `admin_token` cookie, redirect to `/admin/login`
External Unsplash/Google URLs in `<Image>` components	`next/image` domain error	Add to `next.config.mjs` remotePatterns, or migrate to own CDN
`Math.random()` in CollectionPage render	Hydration mismatch error	Replace with deterministic data

9.8 Environment Separation

Variable	local	staging	production
`APP_DEBUG`	true	false	**false**
`DB_HOST`	localhost	private IP	RDS / managed DB
`CACHE_DRIVER`	file	redis	redis
`QUEUE_CONNECTION`	sync	redis	redis
`SESSION_SECURE_COOKIE`	false	true	true
`SANCTUM_STATEFUL_DOMAINS`	localhost:3000	staging.bloomparfums.ma	bloomparfums.ma

`APP_DEBUG=true` on production is a critical vulnerability — Laravel prints full stack traces including environment variables in API error responses.

---

10. Final Recommendations

10.1 Critical Issues — Fix Before Any Feature Work

Priority	Issue	Action
P0	`tymon/jwt-auth` with client-readable cookie	Replace with `laravel/sanctum` + HttpOnly cookie
P0	`/register` page with public registration route	Remove both the page and the backend route
P0	`/dashboard` with loyalty points, Orders, Wishlist stats	Repurpose as admin dashboard or remove
P0	`Math.random()` in CollectionPage sidebar histogram causes hydration error	Replace with fixed data array
P0	Undefined CSS classes (`brand-600` , `form-input` , `btn-secondary`)	Define in globals.css
P0	`<Link href="/order-status">` used as form submit in track-order	Replace with proper `<form onSubmit>` handler wired to API

10.2 High-Priority Feature Implementation Order

Phase	Deliverable
Phase 1 — Backend Foundations	Migrations for all 13 tables. Seed shipping methods, brands, categories. `Admin` model + Sanctum. Product model extended.
Phase 2 — Public Catalog API	`ProductController` , `BrandController` , `CategoryController` , `ShippingMethodController` with full responses and Redis caching.
Phase 3 — Frontend Wiring (Catalog)	Wire `BestSellers` , `Collection` , `ProductDetail` to real API using SWR. Implement Zustand cart store. Implement cookie wishlist utility.
Phase 4 — Order System	Backend: `OrderService` , `StoreOrderRequest` , `POST /orders` , `GET /orders/{number}/track` . Frontend: wire Checkout form, Success page dynamic content, TrackOrder form, OrderStatus SWR polling.
Phase 5 — Coupon System	`CouponController@validate` , wire to CartDrawer and Checkout coupon input with validation state.
Phase 6 — Review System	`ReviewService` , `POST /reviews` , `GET /products/{slug}/reviews` . Wire ReviewModal submit button. Wire feedback page product list from order.
Phase 7 — Admin Panel	`/admin/login` , `/admin/dashboard` , product CRUD, order status updates, review moderation. Protected by Next.js Middleware.

Phase	Deliverable
Phase 8 — Production Hardening	Security headers, rate limiting, Redis cache, S3 upload for review images, email queue, performance audit.

10.3 Architecture Summary

10.4 What Is Built Well Today

Item	Assessment
Visual design & brand consistency	Excellent — strong, coherent identity
Post-purchase flow (5 pages)	Excellent — complete UI, correct navigation

Item	Assessment
App Router architecture	Correct — proper use of layouts, client/server boundary
`ForceJsonResponse` middleware	Correct pattern, keep as-is
ProductCard component	Well-structured, proper TypeScript interface
CartDrawer UX	Good sliding drawer pattern with SSR guard
ReviewModal interaction	Well-implemented hover/click star rating
FilterModal design	Correct right-drawer pattern matching FilterModal
Auto-slug on Product	Correct booted() hook pattern
Soft deletes on Product	Correct default behavior
`serverFetch<T>()` utility	Good pattern for server-side auth prefetch

10.5 What Must Be Rebuilt or Removed

Item	Decision
`tymon/jwt-auth` library	Replace with `laravel/sanctum`
`User` model (customer auth)	Remove entirely
`/login`, `/register`, `/dashboard` (public)	Remove or repurpose as admin-only
`authService.login/register` (public)	Remove
`lib/auth.ts` `setAuthToken` (client-readable)	Remove — replaced by HttpOnly admin cookie
`Product.category` as free text	Migrate to `category_id` FK
All hardcoded product data across BestSellers, Collection, ProductDetail, CartDrawer	Replace with real API calls
All hardcoded order data in Success, OrderStatus pages	Wire to API response
`WishlistOverlay` component	Consolidate into `/wishlist` page (remove duplicate)
`Array.fill()` wishlist mock	Replace with cookie utility + API validation
Undefined CSS classes in login/dashboard	Fix or reassign to admin UI design system

Full-Stack Architecture Report — Bloom Parfums E-commerce Platform
Version 3.0 | February 25, 2026
Document produced from live codebase scan. All findings are ground-truth — no assumptions.